

TALLER DE DISEÑO DE SOFTWARE 1

Programación Web — Paso a Paso

CLASE

04

Estructuras Repetitivas

for — while — repitiendo instrucciones

Año 2026

Prof. Ing. Daniel Maldonado



Nota sobre esta clase

Este material es para investigación autónoma. No se dicta en clase — lo lees, lo probás en tu compu, y después me lo explicás a mí para que verifiquemos juntos que lo entendiste. No alcanza con "copiar y pegar" el código: tenés que poder explicar por qué funciona cada parte.

Contenido de esta clase

1. ¿Qué es un ciclo o bucle?
2. Estructura del ciclo for
3. Estructura del ciclo while
4. El patrón contador y acumulador
5. Aplicando contador y acumulador — ejemplos completos
6. for vs while — ¿cuándo uso cada uno?
7. Resumen final

1 ¿Qué es un ciclo o bucle?

Hasta ahora, cada línea de código que escribimos se ejecuta una sola vez. Pero hay situaciones donde necesitamos repetir la misma acción muchas veces — sumar 10 edades, cobrar a varios clientes, pedir una contraseña hasta que sea correcta.

Un ciclo (o bucle) es una estructura que repite un bloque de código mientras se cumpla una condición.



Analogía

Pensá en la fila de un supermercado. La cajera atiende a una persona, cobra, y pregunta: "¿queda alguien más en la fila?". Si la respuesta es sí, atiende a la siguiente persona. Repite esa misma acción — atender a un cliente — una y otra vez, hasta que la fila se vacía y no queda nadie esperando. Un ciclo hace exactamente eso con código: repite una acción hasta que se cumple la condición de corte.

Sin ciclos, si quisiéramos pedir 10 edades tendríamos que escribir la misma línea de `prompt()` diez veces, una debajo de la otra. Con un ciclo, escribimos esa línea una sola vez y le decimos al programa que la repita 10 veces.

2 Estructura del ciclo for

¿Cuándo lo usamos?

Usamos `for` cuando sabemos de antemano cuántas veces se va a repetir el ciclo. La cantidad de vueltas es un número conocido antes de empezar — por ejemplo, "repetir algo 5 veces" o "recorrer 7 días de la semana".

Encabezado y cuerpo

Todo ciclo `for` tiene dos partes bien diferenciadas, y es fundamental distinguirlas desde el principio:

- El **ENCABEZADO** — lo que va dentro del paréntesis `( )`. Ahí se define CUÁNTAS veces se va a repetir el ciclo.
- El **CUERPO** — lo que va dentro de las llaves `{ }`. Ahí van las instrucciones que se REPITEN en cada vuelta.

Veámoslo con el ejemplo más simple posible: repetir una frase 5 veces.

```
for (let iterador = 0; iterador < 5; iterador = iterador + 1) {  
  console.log('Estoy aprendiendo programación');  
}
```

■ *Encabezado* — define cuántas veces se repite el ciclo. ■ *Cuerpo* — lo que se repite en cada vuelta.



Escribí este código en tu archivo JavaScript y ejecutalo:

```
for (let iterador = 0; iterador < 5; iterador = iterador + 1) {  
  console.log('Estoy aprendiendo programación');  
}
```

El encabezado — dentro del paréntesis — tiene tres partes separadas por punto y coma (;):

Parte del encabezado	¿Qué hace?	En el ejemplo
Inicialización	Crea la variable iterador y le da un valor inicial. Se ejecuta una sola vez, al arrancar el ciclo.	let iterador = 0
Condición	Se evalúa antes de cada vuelta. Mientras sea true, el ciclo sigue. Cuando da false, el ciclo corta.	iterador < 5
Incremento	Se ejecuta al final de cada vuelta, después de correr el código del cuerpo.	iterador = iterador + 1

Importante

El ciclo arrancó con iterador en 0 y terminó con iterador en 4 — dio 5 vueltas en total, no 6. Esto es porque la condición es `iterador < 5` (menor que 5, no menor o igual). En cuanto iterador llega a valer 5, la condición `5 < 5` da false y el ciclo corta. Este es uno de los errores más comunes al empezar con for — contar mal cuántas vueltas da.

La variable iterador se llama así porque su función es ir marcando en qué vuelta (o "iteración") del ciclo estamos. En muchos lugares vas a ver esta misma variable escrita como `i` — es solo una abreviación de "iterador" que se volvió costumbre entre programadores, pero el concepto es exactamente el mismo.

3 Estructura del ciclo while

¿Cuándo lo usamos?

Usamos while cuando NO sabemos de antemano cuántas veces se va a repetir el ciclo. La cantidad de vueltas depende de una condición que puede cambiar en cada repetición — muchas veces, una decisión del usuario.

Encabezado y cuerpo

El while también tiene sus dos partes, aunque el encabezado es más simple que el del for:

- El ENCABEZADO — lo que va dentro del paréntesis (). Ahí va la condición que se revisa antes de cada vuelta.
- El CUERPO — lo que va dentro de las llaves { }. Ahí van las instrucciones que se repiten, y también la línea que puede cambiar la condición.

A diferencia del for, el while no tiene una inicialización ni un incremento integrados en su encabezado — esas partes las armás vos mismo, afuera y adentro del cuerpo. Veamos el mismo ejemplo de repetir una frase, pero ahora sin saber de antemano cuántas veces se va a repetir:

```
let continua = 'SI';

while (continua === 'SI') {
  console.log('Estoy aprendiendo programación');
  continua = prompt('¿Continúo? (SI/NO)');
}
```

■ *Encabezado — define cuántas veces se repite el ciclo.* ■ *Cuerpo — lo que se repite en cada vuelta.*

 **Escribí este código en tu archivo JavaScript y ejecutalo:**

```
let continua = 'SI'; // arranca en 'SI' para que el ciclo entre la primera vez

while (continua === 'SI') {
  console.log('Estoy aprendiendo programación');
  continua = prompt('¿Continúo? (SI/NO)');
}
```

Importante

La variable continua tiene que cambiar DENTRO del cuerpo del ciclo — por eso la línea `continua = prompt(...)` está adentro de las llaves. Si esa línea no estuviera, o estuviera afuera del ciclo, la condición `continua === 'SI'` nunca cambiaría y el ciclo se repetiría para siempre. Esto se llama ciclo infinito y es uno de los errores más comunes con while.

4 El patrón contador y acumulador

Ya sabés cómo repetir código con `for` y con `while`. Ahora vamos a usar esa repetición para algo útil: sumar valores y contar cuántas veces pasó algo. Para eso vas a usar dos variables muy comunes en casi todos los ejercicios de esta clase.

Acumulador

Es una variable que va guardando un total a medida que el ciclo se repite. Siempre arranca en 0 (si acumula una suma) o en 1 (si acumula un producto).

```
let sumaEdades = 0; // arranca en 0 porque todavía no sumamos nada

// en cada vuelta del ciclo:
sumaEdades = sumaEdades + edadIngresada;
```

Cada vuelta, el acumulador toma su valor actual y le suma algo nuevo. Al final del ciclo, el acumulador tiene el total de todas las vueltas juntas.

Contador

Es una variable que cuenta cuántas veces pasó algo. Arranca en 0 y en cada vuelta se le suma 1.

```
let cantidadAlumnos = 0; // arranca en 0

// en cada vuelta del ciclo:
cantidadAlumnos = cantidadAlumnos + 1;
```

⚠ Importante

No confundas el contador con la variable iterador del `for`. El iterador controla cuántas vueltas da el ciclo — es parte del encabezado. El contador es una variable que vos armás, adentro del cuerpo, para contar algo específico del problema (por ejemplo, cuántos clientes pagaron con tarjeta). A veces coinciden, a veces no.

5 Aplicando contador y acumulador

Ahora sí, juntamos todo: la estructura del ciclo (encabezado + cuerpo) más el acumulador y el contador.

Ejemplo con `for` — cantidad conocida de antemano

Un profesor tiene 10 alumnos y quiere saber el promedio de sus edades. Como sabemos que son exactamente 10, usamos `for`.

 **Escribí este código en tu archivo JavaScript y ejecutalo:**

```
{
  /*
    Acumulador: va a ir guardando la suma total.
    Arranca en 0 porque todavía no sumamos nada.
  */
  let sumaEdades = 0;
```

```
/*
  Encabezado: el ciclo se repite exactamente 10 veces.
*/
for (let iterador = 0; iterador < 10; iterador = iterador + 1) {
  /*
    Cuerpo: esto es lo que se repite en cada vuelta.
  */
  let edad = Number(prompt('Ingrese la edad del alumno ' + (iterador +
1)));

  sumaEdades = sumaEdades + edad;
}

let promedio = sumaEdades / 10;

console.log('Suma total de edades: ' + sumaEdades);
console.log('Promedio de edad del curso: ' + promedio);
}
```

Notá que acá no hace falta un contador aparte — la variable `iterador` ya nos dice cuántas vueltas dio el ciclo (siempre 10, porque así lo definimos en el encabezado). Un contador aparte se vuelve necesario cuando no sabemos ese número de antemano, como en el siguiente ejemplo.

Ejemplo con while — cantidad desconocida

El mismo problema, pero ahora el profesor no sabe cuántos alumnos va a cargar. Como no conocemos el número de antemano, usamos while, y esta vez sí necesitamos un contador propio.

 **Escribí este código en tu archivo JavaScript y ejecutalo:**

```
{
  let sumaEdades = 0;
  let cantidadAlumnos = 0;  // acá SÍ necesitamos un contador aparte,
                             // porque no sabemos cuántas vueltas van a
  ser

  let continua = 'SI';

  while (continua === 'SI') {
    let edad = Number(prompt('Ingrese la edad del alumno:'));

    sumaEdades = sumaEdades + edad;
    cantidadAlumnos = cantidadAlumnos + 1;

    continua = prompt('¿Desea continuar? (SI/NO)');
  }

  let promedio = sumaEdades / cantidadAlumnos;

  console.log('Cantidad de alumnos cargados: ' + cantidadAlumnos);
  console.log('Promedio de edad: ' + promedio);
}
```

Acá el contador cantidadAlumnos es indispensable, porque no hay ningún iterador que nos diga de antemano cuántas vueltas dio el ciclo. Es la única forma de saberlo al final, y además es lo que necesitamos para calcular el promedio.

6 for vs while — ¿cuándo uso cada uno?

Estructura	¿Cuándo la uso?	Ejemplo de uso
for	Cuando sé de antemano cuántas veces se repite	Cargar las edades de 10 alumnos (número fijo)
while	Cuando NO sé de antemano cuántas veces se repite	Cargar edades hasta que el usuario diga que no quiere seguir



Analogía

Si tenés que atender a 10 clientes que ya reservaron turno, usás for — sabés el número exacto antes de empezar. Si tenés que atender a la fila del supermercado "hasta que no quede nadie esperando", usás while — no sabés cuántas personas van a ser, depende de una condición que se cumple en algún momento imprevisible.

La pregunta que te tenés que hacer siempre

Antes de escribir el ciclo, preguntate: ¿yo ya sé el número exacto de vueltas antes de arrancar?

- Si la respuesta es SÍ → for
- Si la respuesta es NO, porque depende de una decisión, una búsqueda o una condición que se va a cumplir en algún momento indeterminado → while

**Resumen — Todo lo que aprendiste hoy**

Concepto	¿Qué es?
Ciclo / bucle	Estructura que repite un bloque de código mientras se cumple una condición.
Encabezado	La parte del ciclo dentro de (). Define cuándo y cuántas veces se repite.
Cuerpo	La parte del ciclo dentro de { }. Contiene las instrucciones que se repiten.
Iterador	Variable del for (a veces abreviada como i) que controla las vueltas del ciclo.
Acumulador	Variable que va guardando un total a medida que el ciclo se repite. Arranca en 0.
Contador	Variable que cuenta cuántas veces pasó algo. Arranca en 0 y suma 1 en cada vuelta.
for	Se usa cuando se conoce de antemano la cantidad exacta de vueltas.
while	Se usa cuando la cantidad de vueltas depende de una condición dinámica.
Variable bandera	Variable (como continua) que controla la condición de un while, y que tiene que cambiar dentro del cuerpo.
Ciclo infinito	Error que ocurre cuando la condición de un while nunca llega a ser false.

**Antes de la devolución con el profesor**

Vas a tener que poder explicar, con tus propias palabras y sin mirar este documento: (1) qué diferencia hay entre el encabezado y el cuerpo de un ciclo, en for y en while; (2) por qué en un ejercicio con for a veces no hace falta un contador aparte, y en un ejercicio con while casi siempre sí; (3) qué pasa si te olvidás de actualizar la variable bandera dentro de un while; (4) cómo decidís, frente a un problema nuevo, si te conviene for o while.